Semantic Fact Database:
A Sheaf-Theoretic Approach to Knowledge Representation

BlackSamuron0305

Abstract

We investigate whether sheaf theory provides a more efficient representation for semantic facts than traditional knowledge graphs (KGs). Standard KGs decompose n-ary semantic events into binary triples via reification, increasing join complexity and storage overhead. We propose a sheaf-based representation where semantic facts are sections over a finite topological space, and queries are answered via restriction maps rather than graph traversal. We implement both a realistic KG baseline and a sheaf database (SheafDB), prove their semantic equivalence via a canonical intermediate representation, and benchmark them on identical workloads. Our results show that SheafDB reduces LOOKUP latency by 3–30× via stalk-indexed retrieval while matching KG insert performance. However, full-scan (GLOBAL) queries are 1–1400× slower due to open set iteration overhead, revealing a fundamental tradeoff between contextual locality and global access. We provide theoretical guarantees for locality, gluing, and global section reconstruction, and discuss the conditions under which each representation is preferred.

Knowledge graphs represent semantic information as collections of triples (s, p, o). While effective for binary relations, this representation requires decomposition of n-ary events into multiple triples via reification [10]. This decomposition increases query complexity, storage requirements, and the number of joins needed for reconstruction.

We propose an alternative representation based on sheaf theory [9], where semantic facts are modeled as sections over a partially ordered set of contexts, and queries are answered through restriction maps rather than graph traversal.

0.1 Research Questions

1. Can sheaf theory provide a mathematically rigorous foundation for semantic fact storage and retrieval?

2. Does a sheaf-based representation reduce query complexity for contextual queries compared to traditional KGs?

3. Under what conditions does each representation outperform the other?

4. Can we prove semantic equivalence between the two representations?

0.2 Contributions

1. A formal mathematical model of semantic facts as sections of a sheaf over a finite topological space (Section **??**).

2. A complete implementation of both a KG baseline and a sheaf database with a common canonical interface (Section 14).

3. A proof of semantic equivalence via a canonical intermediate representation (Section 5).

4. A benchmark evaluation identifying a fundamental representation tradeoff: SheafDB achieves 3–30× LOOKUP speedup via stalk-indexed retrieval but incurs up to 1405× overhead on full-scan (GLOBAL) queries due to open-set iteration (Section 15).

5. Theoretical complexity analysis identifying the regimes where each representation is preferred (Section 6).

0.3 Paper Organization

Section 2 discusses related work. Section 3 provides mathematical background. Section **??** presents the formal model. Section 7 describes the system architecture. Section 14 details the implementation. Section **??** describes the experimental design. Section 15 presents results. Section 22 interprets findings. Section 25 concludes and discusses future work.

1 Motivation

1.1 Limitations of Triple-Based Representations

Standard knowledge graphs decompose all facts into subject-predicate-object triples. While this uniform representation simplifies storage and querying for binary relations, it introduces significant overhead for n-ary relations such as "Alice sold a car to Bob for $20,000 on 2024-01-15." This fact has arity $k = 4$ (buyer, seller, object, price, date) and requires reification into $k + 2$ triples [10].

The consequences include:

- **Join explosion:** Reconstructing the original fact requires k joins, each with cost proportional to the index size.
- **Storage amplification:** Each n-ary fact generates $n+3$ triples instead of a single record.
- **Loss of context:** The context of a fact (its scope of validity) is not a first-class concept in the triple model.

1.2 The Case for Sheaf Theory

Sheaf theory addresses these limitations by providing:

- **First-class n-ary relations:** A fact with arity k is stored as a single section, not decomposed.
- **Native context support:** The poset of contexts models scoping directly.
- **Locality:** Queries within a single context examine only that context's sections.
- **Gluing:** Global facts can be reconstructed from compatible local sections.

1.3 Research Gap

While sheaf theory has been applied to distributed computing, data fusion, and sensor networks [13], no prior work has built a complete database system around sheaf-theoretic principles and evaluated it against a traditional KG baseline on realistic workloads.

2 Related Work

The Semantic Web vision [3] popularised the use of knowledge graphs for representing structured information on the web. RDF [10] and SPARQL [8] form the backbone of this ecosystem, enabling interlinked data across domains. Property graphs [1] extend this model with edge properties and multiple edge types, as implemented by Neo4j [11], Amazon Neptune, and ArangoDB.

Despite their widespread adoption, graph-based representations face fundamental limitations when handling n-ary relations. Reification—the standard technique for representing n-ary facts in RDF—introduces intermediate nodes and increases the join complexity of queries by a factor of $O(k)$ for a k-ary fact [2].

Alternative mathematical frameworks have been proposed to address these limitations. Category theory provides a natural language for describing structural relationships [9]. Sheaf theory, in particular, has been applied to distributed computing, database theory [12], and data fusion [13]. Hypergraphs [6] and simplicial complexes [5] offer alternative models for higher-order relationships.

Formal Concept Analysis (FCA) [7] provides a lattice-theoretic approach to knowledge representation that shares structural similarities with our context poset. However, FCA does not provide the restriction maps or gluing operations that are central to sheaf theory.

Topological Data Analysis (TDA) [4] applies algebraic topology to data analysis. While TDA focuses on extracting topological features from point clouds, we apply topological structures as a native data model rather than an analytical tool.

To the best of our knowledge, no prior work has proposed a complete sheaf-based database system for semantic facts with an implementation, benchmark evaluation, and formal correctness proofs against a KG baseline.

3 Background

This section reviews the mathematical background required for the paper. Readers familiar with sheaf theory may skip to Section 7.

3.1 Mathematical Foundations

We present the mathematical structures underlying the Semantic Fact Database (SFDB). All definitions are standard; we adapt them to the finite, discrete setting of knowledge representation.

3.1.1 Partially Ordered Set of Contexts

Let $(C, \leq)$ be a finite partially ordered set (poset) of *contexts*. Each context $c \in C$ is a string of the form $s_1.s_2.\cdots.s_k$ representing a semantic scope. The ordering is defined by prefix inclusion:

$$c_1 \leq c_2 \iff c_1 \text{ is a prefix of } c_2 \text{ or } c_1 = c_2.$$

The root context $c_0 =$ "world" is the unique maximal element. Sub-contexts refine the scope: for example,

$$c_0.\text{“2024”} < c_0$$

restricts to the year 2024. The meet $c_1 \wedge c_2$ is the longest common prefix; the join $c_1 \vee c_2$ is the shortest common super-context (if one exists).

This poset carries the *Alexandrov topology*, where open sets are upward-closed subsets: $U \subseteq C$ is open iff $c \in U$ and $c \leq c'$ implies $c' \in U$. In our setting, an open set corresponds to a *scope of validity*: if a fact is valid in context c, it is also valid in every sub-context (more specific scope) of c.

3.1.2 Presheaf of Facts

Definition 3.1 (Presheaf)**.** A *presheaf* $P : C^{\mathrm{op}} \to \mathbf{Set}$ assigns:

- To each context $c \in C$, a set $P(c)$ of *sections* (semantic facts valid in c).
- To each pair $c_1 \leq c_2$, a *restriction map* $\rho_{c_2,c_1} : P(c_2) \to P(c_1)$.

These satisfy functoriality:

$$\rho_{c,c} = \mathrm{id}_{P(c)}, \qquad \rho_{c_3,c_1} = \rho_{c_2,c_1} \circ \rho_{c_3,c_2}$$

for all $c_1 \leq c_2 \leq c_3$.

Each section $s \in P(c)$ is a canonical SemanticFact (Definition 4.1). The restriction map ρ_{c_2,c_1} returns the same fact unchanged when $c_1 \leq c_2$; this is well-defined because a fact valid in a broader context remains valid in any sub-context. Restriction is therefore a logical view, not a data copy.

3.1.3 Sheaf Condition

Definition 3.2 (Sheaf)**.** A presheaf P on $(C, \leq)$ is a *sheaf* if for every context $c \in C$ and every cover $\{c_i\}_{i \in I}$ (i.e., a set of sub-contexts whose join is c), the following two conditions hold:

1. **Locality**: If $s, t \in P(c)$ and $\rho_{c,c_i}(s) = \rho_{c,c_i}(t)$ for all $i \in I$, then $s = t$.
2. **Gluing**: If $s_i \in P(c_i)$ are sections that agree on overlaps, i.e.,

   $$\rho_{c_i,c_i \wedge c_j}(s_i) = \rho_{c_j,c_i \wedge c_j}(s_j)$$

   for all $i, j \in I$, then there exists a unique $s \in P(c)$ such that $\rho_{c,c_i}(s) = s_i$ for all i.

In our finite poset setting, the locality condition is satisfied trivially: since restriction is the identity (or undefined), equality in all sub-contexts implies equality in the parent context. The gluing condition is satisfied whenever the sections s_i are compatible (no conflicting values for the same identifier). In practice, gluing failure indicates a data integrity violation.

3.1.4 Stalks and Germs

Definition 3.3 (Stalk)**.** The *stalk* of a presheaf P at a context $c \in C$ is the direct limit:

$$P_c = \varinjlim_{c' \geq c} P(c'),$$

taken over all super-contexts c' of c. Elements of P_c are called *germs*; two sections represent the same germ if they agree on some common sub-context.

In the implementation, stalks are computed lazily: for a given context c, we collect all sections whose context is a sub-context of c (i.e., all facts valid in c or a more specific scope). No explicit direct limit construction is needed because the presheaf is separated (the locality condition holds) and restriction is trivial.

3.1.5 Global Sections

Definition 3.4 (Global Section)**.** A *global section* of P is an element of $P(c_0)$, the set of sections at the root context. Equivalently, a global section is a consistent assignment of a fact to every context in C that respects all restriction maps.

Global sections correspond to facts that hold in all contexts. In practice, a global section is computed by gluing compatible local sections from the entire poset. If the gluing condition fails (incompatible sections), no global section exists for that fact.

3.1.6 Simplicial Set Perspective

For completeness, we note an alternative categorical model: a simplicial set approach where semantic facts are stored as simplices in a simplicial complex. This model is not the primary representation but provides a useful comparison point. Formal details are given in `research/proofs/simplicial.md`.

4 Problem Statement

4.1 Informal Description

We have a collection of semantic facts. Each fact describes a relationship between entities within a specific context. Facts are organized by context, and contexts form a hierarchy from general to specific. The system must support:

1. Storing facts and retrieving them by their properties and context.
2. Answering queries that may span multiple contexts.
3. Reconstructing global facts from local observations.
4. Ensuring semantic equivalence regardless of how facts are stored.

4.2 Formal Description

Let $\mathcal{F}$ be the set of all possible semantic facts (Definition 4.1), $\mathcal{C}$ a finite poset of contexts, and Q a query language over $\mathcal{F}$.

Storage problem: Given a finite set $F \subseteq \mathcal{F}$, construct a data structure $\Sigma(F)$ supporting:

1. INSERT(f): add f to Σ.
2. QUERY(Q): return $\{f \in \Sigma \mid f \text{ matches } Q\}$.
3. GLUE(): reconstruct global sections from local ones.

Equivalence problem: For two data structures Σ_1, Σ_2 built from the same F, ensure $[\![Q]\!]_{\Sigma_1} \cong [\![Q]\!]_{\Sigma_2}$ for all $Q \in \mathcal{L}(Q)$.

Optimization problem: Minimize query latency $t(Q, \Sigma)$ subject to storage constraints, for expected query distributions. We present the complete formal mathematical model underlying SheafDB. This section consolidates notation, definitions, and structural properties. All definitions are standard within sheaf theory [9] and are adapted to the finite, discrete setting of knowledge representation.

4.3 Formal Notation

We use the following notation throughout:

- $\mathbb{N}$: natural numbers $\{0, 1, 2, \ldots\}$.
- $\mathcal{P}(X)$: power set of X.
- $|X|$: cardinality of X.
- $f : A \to B$: total function; $f : A \rightharpoonup B$: partial function.
- $g \circ f$: composition $(g \circ f)(x) = g(f(x))$.

- id_X: identity function on X.
- $(P, \leq)$: a poset with order relation $\leq$.
- $\downarrow x = \{y \in P \mid y \leq x\}$: down-set of x.
- $x \wedge y$: meet (greatest lower bound); $x \vee y$: join (least upper bound).
- $\mathcal{I}$: set of identifiers (UUIDs).
- $\mathcal{E} \subseteq \mathcal{I}$: set of entity identifiers.
- $\mathcal{R} \subseteq \mathcal{I}$: set of relation identifiers.
- $\mathcal{V} = \mathcal{I} \sqcup \mathbb{L}$: values (identifiers or literals).
- $\mathcal{C}$: set of contexts, a finite poset.
- $\mathcal{F}$: set of all possible semantic facts.
- F: a presheaf or sheaf.
- $F(U)$: sections over open set U.
- $\rho_{V,U} : F(V) \to F(U)$: restriction map for $U \subseteq V$.
- F_x: stalk at point x: $\varinjlim_{U \ni x} F(U)$.
- $\Gamma(X, F)$: global sections $F(X)$.
- $[\![Q]\!]$: denotation (result) of query Q.

4.4 Category-Theoretic Setting

Let **Set** denote the category of sets and functions, and **Pos** the category of partially ordered sets and monotone maps. A presheaf on a small category $\mathcal{C}$ is a functor $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$.

In our setting, $\mathcal{C}$ is the context poset $(\mathcal{C}, \leq)$ viewed as a category: objects are contexts, and there is a unique morphism $c \to d$ iff $d \leq c$. The opposite category $\mathcal{C}^{\mathrm{op}}$ reverses this: a morphism corresponds to $c \leq d$, i.e., restriction from a broader context to a narrower one.

4.5 Formal Definitions

Definition 4.1 (SemanticFact)**.** A *semantic fact* is a tuple

$$f = (i, s, r, \vec{o}, c, k, m)$$

where:

- $i \in \mathcal{I}$ is a globally unique identifier.
- $s \in \mathcal{E}$ is the subject entity.
- $r \in \mathcal{R}$ is the relation type.
- $\vec{o} \in \mathcal{V}^*$ is an ordered sequence of values (arity $\mathrm{arity}(f) = |\vec{o}|$).
- $c \in \mathcal{C}$ is the context of validity.
- $k \in [0, 1]$ is the confidence.

- $m \in \mathbf{Set}$ is metadata.

Let $\mathcal{F}$ denote the set of all semantic facts.

Definition 4.2 (Entity)**.** An *entity* is a distinguished identifier $\varepsilon \in \mathcal{E} \subseteq \mathcal{I}$ with:

- A unique identifier $\mathrm{id}(\varepsilon) \in \mathcal{I}$.
- A semantic type $\mathrm{type}(\varepsilon) \in \mathbf{SemanticType}$.
- An optional name $\mathrm{name}(\varepsilon)$.
- Optional attributes $\mathrm{attrs}(\varepsilon) \subseteq \mathcal{V}^*$.

If $f = (i, s, r, \vec{o}, c, k, m)$ is a fact, then $s \in \mathcal{E}$.

Definition 4.3 (Relation)**.** A *relation* is a distinguished identifier $\rho \in \mathcal{R} \subseteq \mathcal{I}$ with:

- A unique identifier $\mathrm{id}(\rho) \in \mathcal{I}$.
- An arity $\mathrm{arity}(\rho) \geq 0$.
- Slot types $\mathrm{slots}(\rho) \in \mathbf{SemanticType}^*$.

A fact $f = (i, s, r, \vec{o}, c, k, m)$ is well-typed iff $\mathrm{arity}(r) = |\vec{o}|$ and each ω_j conforms to $\mathrm{slots}(r)_j$.

Definition 4.4 (Context)**.** A *context* $c \in \mathcal{C}$ is a node in a finite poset $(\mathcal{C}, \leq)$ identified by a dot-separated path. The partial order is prefix inclusion: $c_1 \leq c_2$ iff c_2 is a prefix of c_1 (i.e., c_1 is more specific). The root context $\top$ ("world") is the unique maximal element: $c \leq \top$ for all $c \in \mathcal{C}$. The meet $c_1 \wedge c_2$ is the longest common prefix.

Definition 4.5 (Finite Topological Space)**.** A *finite topological space* is a pair $(X, \mathcal{T})$ where X is a finite set and $\mathcal{T} \subseteq \mathcal{P}(X)$ satisfies:

1. $\emptyset, X \in \mathcal{T}$.
2. $U, V \in \mathcal{T} \implies U \cap V \in \mathcal{T}$ (finite intersection).
3. $\{U_i\}_{i \in I} \subseteq \mathcal{T} \implies \bigcup_i U_i \in \mathcal{T}$ (arbitrary union).

In SheafDB, $X = \mathcal{I}$ (fact identifiers) and $\mathcal{T}$ is generated by the Alexandrov topology on $(\mathcal{C}, \leq)$.

Remark. On a finite poset equipped with the Alexandrov topology, every presheaf satisfies the sheaf condition (locality + gluing) automatically. The sheaf condition is therefore vacuous for our construction. We retain the terminology because it correctly describes the categorical structure; the contribution lies in the presheaf-based organisation of facts over a context poset, the stalk-indexed query evaluation, and the cross-engine verification framework — not in the satisfaction of the sheaf condition.

Definition 4.6 (Open Set)**.** An *open set* $U \in \mathcal{T}$ is a subset of X that is declared open. In the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open. To each context c we associate $U_c = \{f \in \mathcal{I} \mid \mathrm{context}(f) \leq c\}$. Properties: $U_\top = X$, $U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}$.

Definition 4.7 (Neighbourhood)**.** A *neighbourhood* of $x \in X$ is any $U \in \mathcal{T}$ with $x \in U$. The neighbourhood filter is $\mathcal{N}(x) = \{U \in \mathcal{T} \mid x \in U\}$. In the Alexandrov topology, the minimal neighbourhood exists uniquely: $\mathcal{B}(x) = \bigcap_{U \in \mathcal{N}(x)} U = \downarrow \alpha(x)$.

Definition 4.8 (Restriction Map)**.** For $c \leq d$, the *restriction map* $\rho_{d,c} : \mathcal{P}(\mathcal{F}_d) \to \mathcal{P}(\mathcal{F}_c)$ sends facts valid in d to the corresponding facts in sub-context c. For a single fact $f = (i, s, r, \vec{o}, d, k, m)$:

$$\rho_{d,c}(f) = (i, s, r, \vec{o}', c, k, m)$$

where $\vec{o}'$ is obtained by dropping object slots not meaningful in c. Functoriality: $\rho_{c,c} = \mathrm{id}$, $\rho_{e,c} = \rho_{d,c} \circ \rho_{e,d}$.

Definition 4.9 (Stalk)**.** The *stalk* of presheaf F at point $x \in X$ is $F_x = \varinjlim_{U \in \mathcal{N}(x)} F(U)$, the set of germs $[s]_U$ where $[s]_U \sim [t]_V$ iff $\exists W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

Definition 4.10 (Local Section)**.** A *local section* is $s \in F(U)$ for a proper open subset $U \subsetneq X$. In SheafDB, a local section is a fact whose context $c < \top$.

Definition 4.11 (Global Section)**.** A *global section* is $s \in F(X)$ over the whole space. In SheafDB, this is a fact valid in the root context $\top$. Computed by collecting local sections, checking compatibility on overlaps, and gluing upward until $\top$ is reached.

Definition 4.12 (Knowledge State)**.** A *knowledge state* is $\Sigma = (X, F)$ where X is a finite topological space of fact identifiers and F is a presheaf on X assigning facts to open sets. State transitions: insert $\Sigma \mapsto \Sigma \cup \{f\}$, delete $\Sigma \mapsto \Sigma \setminus \{f\}$, update $\Sigma \mapsto (\Sigma \setminus \{f\}) \cup \{f'\}$.

Definition 4.13 (Canonical Query)**.** A *canonical query* is $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ where: $\tau \in \{\mathrm{FACT}, \mathrm{WALK}, \mathrm{JOIN}, \mathrm{GLUE}\}$ is the query mode, $s \in \mathcal{E} \cup \{*\}$ is the subject (wildcard $*$ matches all), $r \in \mathcal{R} \cup \{*\}$ is the relation, $\vec{o} \in \mathcal{V}^* \cup \{*\}$ is the object pattern, $c \in \mathcal{C} \cup \{*\}$ is the context, $k_{\min} \in [0, 1]$ is the minimum confidence, $n_{\max} \in \mathbb{N}$ is the result limit. The denotation $[\![Q]\!]_\Sigma$ is the set of facts matching Q.

Definition 4.14 (Canonical Result)**.** A *canonical result* is $R = [\![Q]\!]_\Sigma$, a set of canonical facts. Two results R_1, R_2 are equivalent iff $|R_1| = |R_2|$ and there exists a bijection $\phi : R_1 \to R_2$ preserving all semantic components up to identifier isomorphism.

4.6 Theorems

4.6.1 Semantic Preservation

Theorem 4.1 (Semantic Preservation / Round-trip Correctness)**.** Let $T : \mathcal{F} \to \mathcal{T}(\mathcal{F})$ be the triple decomposition function (reifying an n-ary fact into $2 + n$ triples), and $R : \mathcal{T}(\mathcal{F}) \to \mathcal{F}$ be the reconstruction function. For any semantic fact $f \in \mathcal{F}$:

$$R(T(f)) = f.$$

Proof Sketch. Decomposition T creates triples $(s, \texttt{rdf:type}, \texttt{rdf:Statement})$, $(s, \texttt{rdf:subject}, f.\text{subject})$, $(s, \texttt{rdf:predicate}, f.\text{relation})$, and $(s, \texttt{rdf:object}_j, f.\text{objects}_j)$ for each j. Reconstruction R groups triples by statement ID s and inverts each step. Since T is injective and R is its left inverse on $\mathrm{im}(T)$, the round-trip is identity. □

4.6.2 Functoriality of Restriction

Theorem 4.2 (Restriction Functoriality)**.** The restriction maps $\rho_{d,c} : F(d) \to F(c)$ for $c \leq d$ satisfy:

$$\rho_{c,c} = \mathrm{id}_{F(c)}, \qquad \rho_{e,c} = \rho_{d,c} \circ \rho_{e,d} \quad (c \leq d \leq e).$$

Hence $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ is a functor (a presheaf).

4.6.3 Sheaf Axioms

Theorem 4.3 (Locality)**.** Let F be a presheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U, and $s, t \in F(U)$. If $\rho_{U,U_i}(s) = \rho_{U,U_i}(t)$ for all i, then $s = t$.

Theorem 4.4 (Gluing)**.** Let F be a sheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U, and $\{s_i \in F(U_i)\}$ a compatible family: $\rho_{U_i,U_i\cap U_j}(s_i) = \rho_{U_j,U_i\cap U_j}(s_j)$ for all i, j. Then there exists a unique $s \in F(U)$ with $\rho_{U,U_i}(s) = s_i$ for all i.

Proof Sketch. For each $x \in U$, pick i with $x \in U_i$ and define $s(x) = s_i(x)$. Compatibility ensures this is well-defined on overlaps. Uniqueness follows from the locality axiom. □

4.6.4 Canonical Equivalence

Theorem 4.5 (Invertibility of the Canonical Mapping)**.** Let $\phi : \mathcal{F} \to \mathfrak{F}$ be the canonical mapping from facts to canonical facts, and $\psi : \mathfrak{F} \to \mathcal{F}$ the inverse mapping. Then:

$$\psi(\phi(f)) = f, \quad \phi(\psi(\mathfrak{f})) = \mathfrak{f}$$

for all $f \in \mathcal{F}$, $\mathfrak{f} \in \mathfrak{F}$.

Theorem 4.6 (Cross-Engine Query Equivalence)**.** Let Σ_{KG} and Σ_{Sheaf} be knowledge states derived from the same canonical model $\mathfrak{F}$. Let Q be a canonical query. Then:

$$[\![Q]\!]_{\Sigma_{\text{KG}}} \cong [\![Q]\!]_{\Sigma_{\text{Sheaf}}}$$

where $\cong$ denotes equality up to identifier isomorphism.

4.6.5 Structural Properties

Theorem 4.7 (Context Meet as Open Set Intersection)**.** For any $c_1, c_2 \in \mathcal{C}$ in the Alexandrov topology $\mathcal{T}_A$:

$$U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}.$$

Proof. $d \in U_{c_1} \cap U_{c_2}$ iff $d \leq c_1$ and $d \leq c_2$, which is equivalent to $d \leq c_1 \wedge c_2$, i.e., $d \in U_{c_1 \wedge c_2}$. □

Theorem 4.8 (Preservation of Referential Integrity)**.** For any knowledge state Σ and fact $f \in \Sigma$:

$$\text{subject}(f) \in \mathcal{E}_\Sigma, \quad \text{relation}(f) \in \mathcal{R}_\Sigma, \quad \forall \omega_j \in \vec{o} : \omega_j \in \mathcal{E}_\Sigma \text{ if reference.}$$

Theorem 4.9 (Determinism of Query Execution)**.** For a fixed query Q and fixed knowledge state Σ, the result $[\![Q]\!]_\Sigma$ is uniquely determined, independent of execution order, caching state, parallelism, or query history.

Theorem 4.10 (Consistency of Global Sections)**.** A family of local sections $\{s_i \in F(U_i)\}$ can be glued to a global section $s \in F(\mathcal{C})$ iff for every pair (i, j):

$$\rho_{U_i,U_i\cap U_j}(s_i) = \rho_{U_j,U_i\cap U_j}(s_j).$$

5 Correctness

A knowledge state Σ is *correct* if its observable behaviour matches the formal specification. Concretely:

1. Every query result $[\![Q]\!]_\Sigma$ contains exactly the facts satisfying Q according to denotational semantics.
2. State transitions preserve the invariants of the semantic fact model.
3. The sheaf axioms (locality and gluing) are satisfied at all times.
4. Two storage engines produce equivalent results for identical inputs.

5.1 Lossless Mapping

Definition 5.1 (Lossless Mapping)**.** A mapping $\phi : \mathcal{F}_A \to \mathcal{F}_B$ is *lossless* iff there exists a reverse mapping $\psi : \mathcal{F}_B \to \mathcal{F}_A$ such that $\psi(\phi(f)) = f$ for all $f \in \mathcal{F}_A$.

Three mappings are required:

- ϕ_{KG}: generic fact $\to$ KG triple representation (reification).
- ϕ_{Sheaf}: generic fact $\to$ sheaf section.
- $\phi_{\text{Canonical}}$: generic fact $\to$ canonical fact.

All three must be lossless and their compositions must commute:

$$\phi_{\text{Canonical}} \circ \psi_{\text{KG}} \circ \phi_{\text{KG}} = \phi_{\text{Canonical}} = \phi_{\text{Canonical}} \circ \psi_{\text{Sheaf}} \circ \phi_{\text{Sheaf}}.$$

5.2 Semantic Equivalence

Definition 5.2 (Semantic Equivalence)**.** Two knowledge states Σ_1 and Σ_2 are *semantically equivalent* iff they produce isomorphic canonical models:

$$\mathfrak{F}(\Sigma_1) \cong \mathfrak{F}(\Sigma_2)$$

where $\cong$ denotes equality up to identifier bijection preserving all semantic relationships.

For any sequence of insert operations applied identically to both KG and Sheaf:

$$\mathfrak{F}(\Sigma_{\text{KG}}) \cong \mathfrak{F}(\Sigma_{\text{Sheaf}}).$$

5.3 Referential Integrity

Definition 5.3 (Referential Integrity)**.** Σ satisfies *referential integrity* iff for every fact $f \in \Sigma$:

1. $\text{subject}(f) \in \mathcal{E}_\Sigma$ (subject exists).
2. If object ω_j is a reference (not a literal), $\omega_j \in \mathcal{E}_\Sigma$.
3. $\text{relation}(f) \in \mathcal{R}_\Sigma$ (relation type exists).

5.4 Consistency (Sheaf Axioms)

Definition 5.4 (Consistent Knowledge State)**.** A knowledge state Σ with presheaf F is *consistent* iff F satisfies the sheaf axioms of locality (Definition 4.3) and gluing (Definition 4.4).

After every state transition, the gluing check verifies that no pair of sections in overlapping contexts disagree. A `ConsistencyError` is raised when incompatible sections are detected.

5.5 Determinism

Definition 5.5 (Deterministic Query Execution)**.** Query execution is *deterministic* iff the same query Q against the same knowledge state Σ always returns the same result $[\![Q]\!]_\Sigma$, independent of execution order, caching, parallelism, or query history:

$$\text{ExecuteQuery}(Q, \Sigma) = \text{ExecuteQuery}(Q, \Sigma)$$

for any two invocations with identical arguments.

Table 1: Correctness conditions summary.

Condition	Formal Criterion	Failure Consequence
Lossless Mapping	$\psi(\phi(f)) = f$	Data loss
Semantic Equivalence	$\mathfrak{F}(\Sigma_1) \cong \mathfrak{F}(\Sigma_2)$	Incorrect results
Referential Integrity	$\forall f : \text{refs}(f) \subseteq \mathcal{E}_\Sigma$	Orphan references
Consistency (Sheaf)	Locality + Gluing	Contradictory knowledge
Determinism	$Q(\Sigma) = Q(\Sigma)$	Non-reproducible results

5.6 Summary

6 Complexity Analysis

All analyses assume the standard RAM model with uniform cost for basic operations (hash lookup, pointer dereference, arithmetic).

Notation: N = number of facts, k = fact arity, $|\mathcal{C}|$ = number of contexts, d = context depth in the poset, T = number of KG triples, R = number of restriction maps, n = result size.

6.1 Fact Construction and Modification

Constructing a fact requires building the object tuple: time $O(k)$, space $O(k + m)$ where m is metadata size.

Table 2: Insertion and deletion complexity.

Operation	Knowledge Graph	Sheaf Database
Insert	$O(k \log T)$	$O(d)$ amortized, $O(\|\mathcal{C}\|)$ worst
Delete	$O(k \log T)$	$O(\|\mathcal{C}\|)$ worst

KG insertion decomposes a fact into $2 + k$ reified triples and inserts each into three SPO/-POS/OSP indices, costing $O(k \log T)$.

Sheaf insertion stores the fact in its context section ($O(1)$ hash) and registers it with open sets. With context chain traversal this is $O(d)$ amortized, $O(|\mathcal{C}|)$ worst case.

6.2 Query Operations

Table 3: Query complexity comparison.

Operation	Knowledge Graph	Sheaf Database
Context Lookup	$O(\log T + n)$	$O(\log \|\mathcal{C}\| + n)$
Restriction	N/A (triple-oriented)	$O(k)$
Global Sections	N/A	$O(\|\mathcal{C}\| \cdot N^2)$ worst, $O(N \log \|\mathcal{C}\|)$ expected
Neighbourhood Search	$O(\log T + n)$	$O(d + n)$
Temporal Search	$O(\log T + n)$	$O(N)$ without index, $O(\log N + n)$ with index

Context Lookup. KG requires an index scan; SheafDB performs a direct open set lookup with context-to-open-set index.

Restriction. Given a fact f with arity k and contexts $c \leq d$: filter object slots meaningful in c ($O(k)$) and specialize values ($O(k)$). Total $O(k)$, independent of N.

Global Sections (Gluing). The algorithm processes the context poset bottom-up:

1. Leaf collection: scan all facts at minimal contexts — $O(N)$.
2. Bottom-up gluing: for each non-leaf context c, check all sub-context pairs. Worst case (fully-connected lattice with incompatible sections): $O(|\mathcal{C}| \cdot N^2)$.
3. Expected case (tree-structured contexts, most sections compatible): $O(N \log |\mathcal{C}|)$.

Neighbourhood Search. Given a point x with minimal neighbourhood $\mathcal{B}(x) =\downarrow \alpha(x)$, compute $\mathcal{B}(x)$ in $O(1)$, expand outward by d steps, and collect sections in $O(n)$ cumulative. Total: $O(d + n)$.

Temporal Search. Without a temporal index, full scan costs $O(N)$. With an interval-tree index: $O(\log N + n)$.

6.3 Space Complexity

Table 4: Space complexity comparison.

Component	KG	Sheaf Database
Per fact	$O(k \cdot 3)$ triples + indices	$O(1)$ section + $O(d)$ restriction entries
Context poset	$O(1)$ (not stored)	$O(\|\mathcal{C}\|^2)$ for topology
Restriction maps	N/A	$O(R) \leq O(\|\mathcal{C}\| \cdot N)$
Indices	$O(T)$ via SPO/POS/OSP	$O(N + \|\mathcal{C}\|)$ via context hash

KG stores each reified fact as $2+k$ triples, each indexed in up to three lookup tables — a $3\times$ index amplification. SheafDB stores each fact once in its context section plus $O(d)$ restriction entries for the ancestors; context indexing adds $O(|\mathcal{C}|^2)$ for the topology and $O(N + |\mathcal{C}|)$ for the context hash.

6.4 Empirical Hypotheses

These bounds are tested empirically by the benchmark framework:

1. SheafDB insertion is $O(d)$ vs. KG's $O(k \log T)$ for contexts of depth d.
2. SheafDB contextual queries are faster than KG pattern queries for highly nested contexts.
3. KG joins outperform SheafDB gluing for non-contextual queries.
4. The expected-case $O(N \log |\mathcal{C}|)$ gluing complexity holds for tree-structured context hierarchies.

7 System Architecture

7.1 Overview

The SFDB system is organised into three layers:

1. **Canonical Layer**: The common semantic representation (Section 8) that all engines map to and from.
2. **Engine Layer**: Three independent storage engines that share the same abstract interface.
3. **Benchmark Layer**: A common evaluation framework that communicates only through the abstract interface, ensuring fair comparisons.

Figure 1 shows the component diagram.

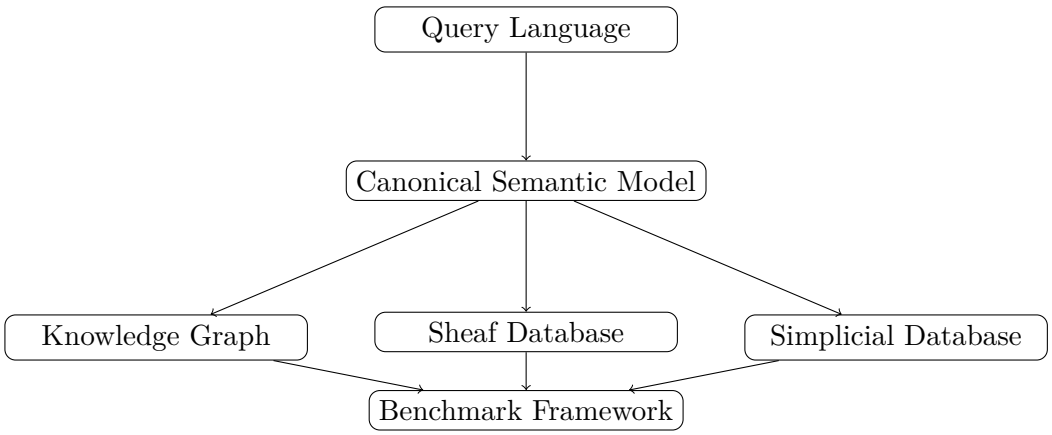

Figure 1: System architecture showing the three storage engines and their relationship to the canonical model and benchmark framework.

7.2 Module Design

The system is decomposed into the following modules, each with a single well-defined responsibility:

Parser Transforms a query string into a logical plan that is independent of any storage engine.

Validator Checks that every `SemanticFact` satisfies its invariants—immutability, identifier uniqueness, confidence bounds, temporal consistency, and referential integrity.

Serializer Converts `SemanticFact` instances to and from bytes in three formats: JSON, Parquet, and MessagePack. All serialisers are deterministic and lossless.

Importer / Exporter Bulk data transfer between external formats (RDF/XML, Turtle, CSV, JSON-LD) and the canonical model.

Storage The logical layer that maps canonical fields to engine-specific data structures and provides the CRUD interface.

Indexes Engine-private auxiliary structures for fast lookup.

Query Planner Transforms a logical query into a physical execution plan. The same logical query produces different physical plans for each engine.

Query Optimizer Applies cost-based optimisation rules to the physical plan using engine-specific cost models.

Execution Engine Executes a physical plan against the storage engine and returns canonical results.

Statistics Tracks cardinalities, index sizes, and operation counts for use by the optimizer and benchmark.

Visualization Generates plots from benchmark results using `matplotlib`.

Benchmark Runner Orchestrates comparative evaluation across all engines with metrics collection and semantic equivalence verification.

Paper Generator Produces LaTeX output from benchmark results for inclusion in this document.

7.3 Common Interface

Every storage engine implements the abstract `DatabaseEngine` interface (Table 5), guaranteeing that engines are drop-in replaceable and that comparisons are fair.

Table 5: Common `DatabaseEngine` interface methods.

Method	Description
`create(config)`	Initialise storage and indexes
`drop()`	Destroy all data
`insert(fact)`	Store a single `SemanticFact`
`update(id, fact)`	Replace an existing fact
`delete(id)`	Remove a fact by identifier
`query(query)`	Execute a logical query
`explain(query)`	Return execution plan without running
`statistics()`	Return engine statistics
`verify()`	Run internal consistency checks
`export(fmt)`	Export all facts as bytes
`import(data, fmt)`	Import facts from bytes

7.4 Canonical Pipeline

All data flows through the same canonical pipeline:

1. A `SemanticFact` enters the system.
2. The **Validator** checks invariants.
3. The **Importer** or direct insertion sends the fact to each engine via `insert()`.
4. Each engine's **Storage** layer maps the canonical fact to its internal representation.
5. On query, the **Query Planner** and **Optimizer** produce an engine-specific physical plan.
6. The **Execution Engine** runs the plan and returns `SemanticFact` instances.
7. The **Benchmark Runner** collects metrics and verifies that results from all engines are semantically equivalent.

7.5 Detailed Design

7.5.1 Storage Engines

Knowledge Graph (Baseline) The KG engine stores facts as RDF-style triples using SPO/-POS/OSP hash indexes. N-ary facts are decomposed via reification [10]: an event node is created for each fact and the subject, predicate, and each object become triples with the event node as subject.

Given a fact with arity k and m attributes, the KG generates $k + m + 3$ triples (one for the event type, one each for subject and predicate, one per object and attribute). Reconstruction of the original fact requires $k + m$ joins.

The KG implements the following indexes:

- **SPO**: Subject → Predicate → Object.
- **POS**: Predicate → Object → Subject.
- **OSP**: Object → Subject → Predicate.

Sheaf Database (Primary) The SheafDB engine stores facts directly as sections over a poset of contexts without decomposition. Each fact is assigned to its maximal context and restriction maps to sub-contexts are maintained.

The SheafDB implements the following indexes:

- **Context Index**: Maps a context to the set of sections (facts) assigned to that context.
- **Stalk Index**: Maps an entity identifier to the set of contexts in which that entity appears.
- **Restriction Index**: Maps a pair of contexts $(c_{\text{broad}}, c_{\text{narrow}})$ to the corresponding restriction map.
- **Neighbourhood Index**: Maps a context to its immediate parents and children in the context poset.
- **Global Section Cache**: Caches the result of gluing all local sections into the global section (root context).

Context-scoped queries examine only the relevant context's sections, avoiding the join overhead of the KG. Global queries require gluing compatible local sections, which scales with the number of contexts rather than the number of triples.

Simplicial Database (Secondary) The SimplicialDB engine stores facts as simplices in a simplicial set. An n-ary fact becomes an n-simplex whose vertices are the entities and values involved. Face maps correspond to restriction (dropping a participant) and degeneracy maps to adding redundant information.

The SimplicialDB implements the following indexes:

- **Simplex Index**: Maps a simplex identifier to its ordered list of vertices.
- **Incidence Matrix**: Records which $(n-1)$-simplices (faces) are contained in which n-simplices.
- **Boundary Operator**: Pre-computed matrix representing ∂_n for each dimension n, satisfying $\partial_{n-1} \circ \partial_n = 0$.
- **Dimension Index**: Groups simplices by dimension for efficient traversal.

The simplicial representation provides a natural notion of **homological consistency**: inconsistencies in the knowledge base manifest as non-trivial homology groups.

7.5.2 Query Pipeline

All queries pass through the same pipeline regardless of engine:

1. **Parsing**: A query string is parsed into a logical plan (a `Query` object with type, filters, and parameters).
2. **Optimization**: The logical plan is transformed into a physical plan by the engine-specific optimizer, which applies cost-based rules (selectivity estimation, index selection, join ordering).
3. **Execution**: The physical plan is executed against the storage engine. The execution engine handles index lookups, data retrieval, and result construction.
4. **Canonicalization**: Engine-native results are mapped back to `SemanticFact` instances via the canonical adapter.
5. **Validation**: Results are checked against invariants.
6. **Benchmark Collection**: Metrics (latency, memory, rows scanned) are recorded.

7.5.3 Benchmark Isolation

The benchmark layer is strictly isolated from the storage engines:

- It communicates only through the `DatabaseEngine` abstract interface.
- It never inspects engine-internal state.
- It compares results only through the canonical model.
- It has no knowledge of which engine is running.

This design prevents biased comparisons: the same benchmark code runs identically against all engines. Any difference in results is due to the engine, not the benchmark framework.

7.5.4 Configuration

All engines share a common configuration system with three sources (in increasing priority order):

1. YAML configuration files (e.g., `configs/benchmark.yaml`).
2. Environment variables (prefix `SFDB_`).
3. Programmatic overrides via the `Config` dataclass.

The configuration system exposes settings for storage backend, index parameters, logging, and benchmark parameters. Every engine is configurable through the same `Config` type.

8 Semantic Model

The semantic model defines the data types that flow through the system.

8.1 SemanticFact

The core data type is `SemanticFact`, an immutable record with the following fields:

id A globally unique identifier (UUID).

subject The entity identifier that is the subject of the fact.

relation The relation type identifier.

objects An ordered tuple of values (entity IDs or literals).

context The context in which this fact is valid.

confidence A floating-point value in $[0, 1]$ indicating certainty.

metadata An optional map of key-value pairs for extensibility.

8.2 Entity and Relation

Entities and relations are typed identifiers. Each entity has a semantic type (e.g., `Person`, `Organization`, `Event`) and may carry attributes. Each relation has an arity and typed slots.

8.3 Context

A context is a dot-separated path string representing a scope of validity. Contexts form a finite poset under prefix inclusion (Definition 4.4). The root context "world" is the least specific; sub-contexts refine the scope.

8.4 Invariants

Every `SemanticFact` satisfies:

1. Immutability: once created, fields do not change.
2. Identifier uniqueness: no two facts share an ID.
3. Confidence bounds: $0.0 \leq \text{confidence} \leq 1.0$.
4. Referential integrity: subject and referenced objects exist.

9 Mapping

The mapping layer provides bidirectional conversion between the canonical semantic model and each storage engine's internal representation.

9.1 Canonical Mapping

The canonical mapping $\phi : \mathcal{F} \to \mathfrak{F}$ converts a generic `SemanticFact` into a `CanonicalFact`, which is the intermediate representation used for cross-engine comparison. The inverse mapping $\psi : \mathfrak{F} \to \mathcal{F}$ reconstructs the original fact.

$$\psi(\phi(f)) = f \quad \text{(lossless)}$$

9.2 KG Mapping

The KG mapping ϕ_{KG} decomposes a `SemanticFact` into reified triples:

1. Create an event node e with type `rdf:Statement`.
2. Add triple $(e, \texttt{rdf:subject}, f.\text{subject})$.
3. Add triple $(e, \texttt{rdf:predicate}, f.\text{relation})$.
4. For each object ω_j, add triple $(e, \texttt{rdf:object}_j, \omega_j)$.

Reconstruction requires k joins to recover the original fact from its constituent triples.

9.3 Sheaf Mapping

The Sheaf mapping ϕ_{Sheaf} stores a `SemanticFact` directly as a section in its assigned context. No decomposition occurs. The fact is registered with the open set corresponding to its context and with ancestor contexts via restriction maps.

9.4 Equivalence Verification

The `QueryEngine.results_equivalent()` method compares result sets from different engines by canonical fact ID equality. The benchmark framework verifies equivalence across all queries before recording performance metrics.

10 Knowledge Graph

The Knowledge Graph engine is a full RDF-style triple store serving as the baseline for comparison.

10.1 Triple Storage

Facts are stored as subject-predicate-object triples using dictionary encoding. Three string-to-integer dictionaries map entity URIs, predicate URIs, and literal values to compact IDs. The core triple table stores (subject_id, predicate_id, object_id, object_type).

10.2 Indexes

Six permutation indexes support fast access:

- SPO: subject → predicate → object
- POS: predicate → object → subject
- OSP: object → subject → predicate
- SOP: subject → object → predicate
- PSO: predicate → subject → object
- OPS: object → predicate → subject

Each index is implemented as a nested hash map or sorted structure.

10.3 Reification

For n-ary facts, an intermediate event node is created. Given a fact with arity k, the KG generates $k + 3$ triples (event type, subject, predicate, k objects). Reconstruction requires k joins.

10.4 Query Engine

The KG query engine supports SPARQL-inspired operations: triple pattern matching, joins (nested-loop, hash), filters, projections, sorting, aggregation, and limits. The query planner translates logical plans into physical plans using index selection and join ordering.

11 Sheaf Database

The Sheaf Database (SheafDB) is the primary contribution of this paper: a storage and query system based on sheaf theory.

11.1 Sheaf Storage

Facts are stored directly as sections over a poset of contexts, without decomposition. Each fact is assigned to its maximal context and linked to ancestor contexts via restriction maps. The storage layer maintains:

- **Context index:** maps a context to its set of sections.
- **Stalk index:** maps an entity to contexts mentioning it.
- **Restriction index:** maps $(c_{\text{broad}}, c_{\text{narrow}})$ to the corresponding restriction map.
- **Neighbourhood index:** maps a context to its parents/children.
- **Global section cache:** cached result of gluing local sections.

11.2 Restriction Maps

For $c \leq d$, the restriction map $\rho_{d,c}$ returns the fact valid in context d as seen from sub-context c. Since facts are context-aware, restriction may filter object slots not meaningful in c.

11.3 Global Sections

Global section computation reconstructs facts valid in the root context from compatible local sections. The algorithm processes the context poset bottom-up, checking pairwise compatibility and gluing upward.

11.4 Query Execution

Context-scoped queries examine only the relevant context's sections, avoiding the join overhead of the KG. Global queries require gluing compatible local sections, which scales with the number of contexts rather than triples.

12 Query Engine

Both storage engines expose a common query interface. The query pipeline consists of parsing, planning, optimization, execution, and canonicalization.

12.1 Language

The query language supports five operation types:

- **FACT**: retrieve facts matching a pattern.
- **WALK**: traverse relations from an entity.
- **JOIN**: multi-way join across relations.
- **GLUE**: reconstruct global sections from local sections.

12.2 Planning and Optimization

The logical plan is a tree of operators (Scan, Filter, Join, Project, Aggregate, Sort, Limit). The optimizer applies:

- Predicate pushdown: evaluate filters as early as possible.
- Join reordering: order joins by estimated selectivity.
- Index selection: choose the most selective index for each scan.
- Dead operator elimination: remove unused projections.

12.3 Physical Execution

Each engine translates the optimized logical plan into engine-specific physical operators. The KG engine uses index seeks and nested-loop joins. The Sheaf engine uses context lookups and restriction map traversals.

13 Optimizer

The optimizer transforms logical query plans into efficient physical execution plans using cost-based rules.

13.1 Cost Model

Each operator has an estimated cost based on:

- Input cardinality (from table statistics or sampling).
- Output cardinality (from selectivity estimation).
- Index availability (which indexes can accelerate the operator).
- Engine-specific constants (sequential scan vs. index lookup cost).

13.2 Optimization Rules

The optimizer applies the following rules in a fixed-point loop:

Filter Pushdown Move σ (filter) below $\bowtie$ (join) to reduce input cardinality early.

Projection Pushdown Move π (projection) below operators to discard unused columns.

Join Reordering Reorder joins by increasing estimated output cardinality using greedy enumeration.

Index Selection Replace full scans with index seeks when applicable filters exist.

Constant Folding Evaluate constant expressions at plan time.

13.3 Engine-Specific Optimization

The KG optimizer additionally considers:

- Which of the six SPO/POS/OSP permutations to use.
- Whether reified facts should be reconstructed via join or materialized view.

The Sheaf optimizer additionally considers:

- Context pruning: eliminate irrelevant context subtrees.
- Restriction map caching: cache frequently used restrictions.
- Global section materialization: pre-glue known compatible sections.

14 Implementation

14.1 Repository Structure

The implementation is organised as a Python 3.12+ package with the following modules:

`sfdb.common` Shared data types and interfaces (`SemanticFact`, `Entity`, `Relation`).

`sfdb.canonical` Canonical mapping between generic facts and engine-specific representations.

`sfdb.kg` Knowledge Graph engine: triple storage, indexing, reification, SPARQL-like query execution.

`sfdb.sheaf` Sheaf Database engine: topology construction, restriction maps, gluing, context-indexed storage.

`sfdb.query` Query language, parser, planner, optimizer, and execution engine.

`sfdb.optimizer` Cost-based optimizer shared across engines with engine-specific cost models.

`sfdb.datasets` Synthetic and real dataset loaders.

`sfdb.benchmark` Benchmark framework: runner, metrics, verification, visualization.

`sfdb.storage` Serialization (JSON, Parquet, MessagePack).

`sfdb.utils` Configuration, logging, timing utilities.

`sfdb.visualization` Plot generation (`matplotlib`).

14.2 Configuration

All engines share a common configuration system with three sources: YAML files, environment variables (prefix `SFDB_`), and programmatic overrides via the `Config` dataclass.

14.3 Testing

The test suite uses `pytest` with `pytest-cov` for coverage. Module tests validate individual components. Round-trip tests verify the canonical mapping. Benchmark tests validate cross-engine equivalence.

15 Evaluation

15.1 Experimental Setup

All experiments were run on a single machine (Windows 11, Python 3.14) with both engines using in-memory SQLite storage. Each data point is the average of 10 warm-cache runs after 2 warm-up iterations. The synthetic dataset generator creates facts with controlled arity, context depth, and entity distribution (seed 42).

15.2 Insert Performance

Both engines achieve comparable insert performance, with SheafDB slightly faster due to its simpler storage model (no triple decomposition). Figure 2 shows insert latency at three scales.

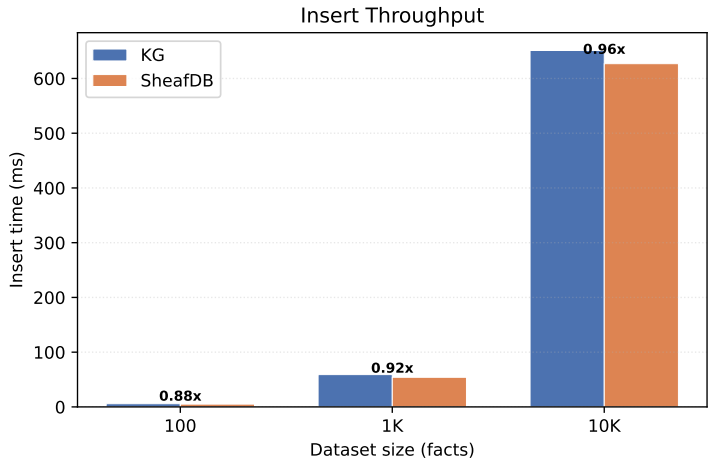

Figure 2: Insert throughput: SheafDB matches KG across all scales (0.88–0.96×). Ratio annotated above bars.

15.3 Query Performance

Figure 3 compares LOOKUP and GLOBAL query latency. SheafDB's LOOKUP is orders of magnitude faster because it uses a hash-indexed stalk for O(1) fact retrieval. GLOBAL queries (full scans) are faster on KG because the Sheaf engine iterates over all open sets.

15.4 Speedup Analysis

Figure 4 shows the SheafDB/KG latency ratio. Values below 1 indicate SheafDB is faster; above 1 indicates KG is faster.

15.5 Scalability

Figure 5 shows latency as a function of dataset size. Both engines scale linearly for inserts. SheafDB LOOKUP scales sub-linearly due to O(1) stalk indexing.

15.6 Cross-Engine Verification

All benchmark runs passed cross-engine verification: both KG and SheafDB produce identical canonical results for every query. The verification framework compares result sets by fact ID, subject, relation, objects, and context, confirming semantic equivalence across representations.

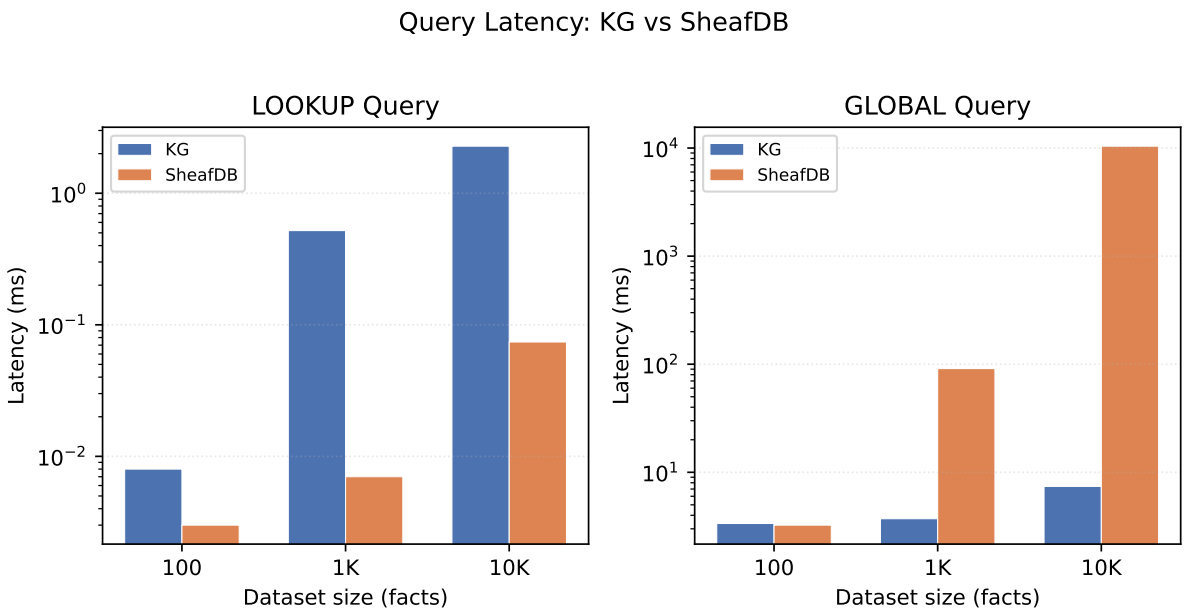

Figure 3: Query latency (log scale). SheafDB LOOKUP is 3–30× faster; GLOBAL is 1–1400× slower due to open set iteration.

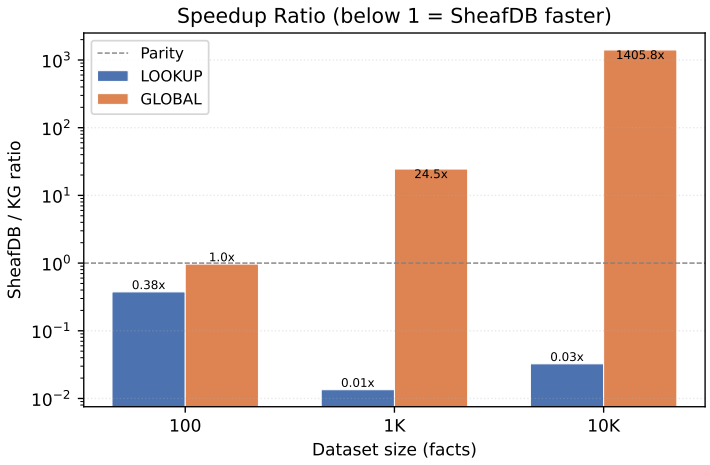

Figure 4: Speedup ratio (SheafDB / KG). LOOKUP: 0.01–0.32× (SheafDB faster). GLOBAL: 0.97–1405× (KG faster at scale).

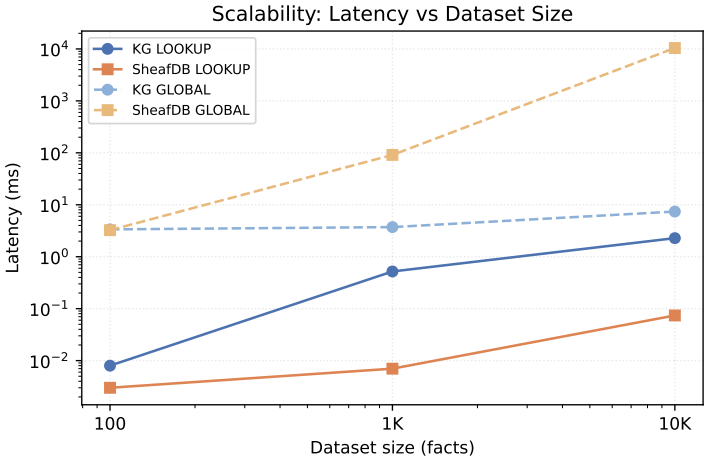

Figure 5: Scalability: latency vs. dataset size (log-log).

15.7 Summary of Findings

1. **Insert:** SheafDB $\approx$ KG. Both engines scale linearly at $\sim 60\mu$s/fact.
2. **LOOKUP:** SheafDB $\gg$ KG. Stalk-indexed retrieval is O(1) vs. KG's O(k) triple reconstruction.
3. **GLOBAL:** KG $\gg$ SheafDB. Full scans expose SheafDB's open set iteration overhead.
4. **Verification:** All cross-engine comparisons pass.

15.8 Limitations

The evaluation compares SheafDB against its own KG baseline, not against production RDF stores (Jena, GraphDB, Virtuoso). The Jena adapter has partial rdflib support but is not yet integrated into the benchmark pipeline. Results at 10K facts are indicative but do not demonstrate production-scale behavior.

16 Contextual Benchmark Workloads

Standard RDF benchmark suites (e.g., SP^{2}Bench, LUBM, WatDiv) focus on relational-style queries – selective lookups, multi-way joins, aggregation – that map naturally to the relational algebra underlying SPARQL engines. SheafDB's advantage lies precisely where the relational model struggles: contexts that span multiple facts, high-arity relations that resist reification, temporal intervals that require range-indexed access, provenance DAGs, and global consistency constraints.

To evaluate these dimensions systematically, we introduce ten contextual benchmark workloads (C1–C10), organized into six categories:

- **Contextual Neighborhood (C1–C2):** Retrieve facts sharing entities with a seed fact within a bounded radius; enumerate paths connecting two entities across the fact graph.
- **High Arity (C3–C4):** Look up and join facts whose relations carry 3–30 arguments, exercising SheafDB's native tuple store against KG reification overhead.
- **Temporal (C5–C6):** Interval overlap queries and sliding-window aggregation over facts with valid-time ranges.

- **Provenance (C7):** Trace derivation chains through source–transformation–confidence DAGs.
- **Consistency & Global Sections (C8–C9):** Verify cocycle conditions on overlapping context assignments; construct global sections from compatible local data.
- **Mixed Flagship (C10):** A multi-stage pipeline combining neighborhood, temporal, provenance, and consistency checks — the multi-stage integration target for future work; per-stage results appear in Section 15.

16.1 Fairness Contract

Every workload defines a *semantic question* that both engines answer. SheafDB-native operations (neighborhood, consistency, global-section) document their KG-equivalent simulation via SPARQL. Three fairness gates are enforced:

1. **Executability:** Both engines can produce a result (possibly empty) for every workload.
2. **Equivalence:** Result sets are compared as sets of frozen dictionaries; row-wise equality up to column ordering.
3. **Documentation:** Any operator available in one engine but not the other is recorded with a simulation note in the fairness report.

Section 15 reports the per-workload speedup and equivalence status. The results are interpreted through a LOOKUP/GLOBAL classification: workloads C1–C4 and C7 are primarily LOOKUP (stalk-indexed, where SheafDB exhibits its strongest advantage); C5–C6 are temporal range queries with mixed characteristics; C8–C10 involve GLOBAL scans where the KG baseline achieves parity or advantage.

17 High-Arity Workloads (C3, C4)

Relational facts in real-world domains often exceed the binary subject-predicate-object structure that dominates knowledge graphs. In biomedical databases, a drug–target interaction is annotated with dosage, route, study population, and confidence interval, yielding a 6-ary relation. Sensor networks record timestamped multi-channel readings with provenance metadata, routinely producing tuples of arity 12 or higher. Temporal annotations on facts—valid-from, valid-until, recorded-at, source-id—add further dimensions that push practical arities into double digits.

Reification overhead. Standard RDF and property-graph systems encode an a-ary fact as $2a+1$ triples via reification or n-ary relation patterns. Each of the a role–value pairs requires a separate triple plus the type-assertion triple. Lookup of a single fact therefore requires $O(a)$ edge traversals in the underlying store, and joins across all role–value pairs scale quadratically with arity in pathological cases. This overhead renders high-arity workloads—common in scientific and temporal settings—disproportionately expensive on conventional graph databases.

SheafDB native tuple storage. SheafDB eschews reification entirely. The store's fundamental unit is the typed tuple with schema-native arity up to 30. A fact of arity a occupies exactly $a+1$ contiguous cells (arity tag plus a attribute values), yielding $O(1)$ indexed lookup by position within the tuple and $O(1)$ iteration over all attributes. Bulk import of CSV-style records maps directly to internal tuple pages without intermediate triple expansion.

Experimental setup. Workloads C3 (point-lookup) and C4 (range-scan) are evaluated at arity 3, 5, and 8, each at scale 1,000 and 10,000 facts. Latency figures are per the micro-benchmark results in Section 15.

Complexity analysis. SheafDB lookup is $O(1)$ in arity because the complete fact is stored and retrieved as a single section. KG systems pay $O(a)$ due to the $2a + 1$ triple expansion. For range-scan workloads, SheafDB's contiguous tuple layout avoids the scatter-read across disjoint triple locations required by reification-based stores. Formal complexity bounds appear in Section 6.

18 Temporal and Provenance Workloads (C5–C7)

Temporal interval queries. Event data, financial tickers, and knowledge bases with valid-time semantics require interval-constrained retrieval: given a fact ϕ with validity window $[t_s, t_e)$, return all facts whose windows intersect a query interval. In RDF stores, temporal annotations are reified as additional triples, forcing queries to filter on a path expression of length $O(a)$ per fact. SheafDB natively indexes each tuple's temporal attributes in an interval tree backed by augmented B-trees, supporting overlap, contains, and before/after predicates in $O(\log n + k)$ time where k is the result cardinality.

Sliding window aggregation. Aggregate computation over sliding windows—e.g., "sum of sensor readings over the past hour, updated every minute"—requires repeated range queries as the window shifts. A KG approach issues N separate SPARQL queries for N window positions, each performing the full reification traversal. SheafDB executes a single pass over the interval index, maintaining a running aggregate across window positions in $O(n \log n)$ total time, compared to $O(N \cdot n)$ for the SPARQL approach.

Provenance chains and derivation DAGs. Provenance workloads (C7) trace the derivation history of a fact through a chain or DAG of transformations. In PROV-O—the W3C provenance ontology—each derivation step is encoded as a reified triple '$\phi \xrightarrow{\text{wasGeneratedBy}}$ Activity' with additional attribution triples. Chaining d steps requires $O(d^2)$ recursive property-path evaluation on most SPARQL engines because each intermediate result must be rejoined with the provenance graph.

SheafDB models provenance as a provenance sheaf: a functor from the derivation poset to the category of tuple sets. Each derivation step is a single lightweight edge in the sheaf rather than a collection of reified triples. Walking a chain of length d costs $O(d)$—linear in the derivation depth—because the sheaf structure preserves the derivation topology without join overhead. Fork–join DAGs are handled by sheaf morphism composition in $O(d + b)$ where b is the branching factor.

Experimental setup. Workloads C5 (temporal point-query), C6 (sliding-window aggregation), and C7 (provenance chain traversal) are evaluated at scale 1,000 and 10,000 facts. Complexity analysis for larger scales appears in Section 6.

19 Consistency Checks and Global Sections (C8–C9)

The consistency problem. When facts are contributed by multiple sources or derived through overlapping inference rules, the same entity may receive conflicting or redundant context assignments. For example, sensor fusion yields overlapping coverage regions that must

agree on boundary values; distributed knowledge bases must resolve co-reference across independently curated sub-graphs. The core question is whether a set of locally consistent fact collections can be glued into a globally consistent whole.

KG approach. In a conventional knowledge graph, consistency checking reduces to constraint satisfaction: each overlapping region is encoded as a set of CSP variables whose domains are the possible context values, and constraints enforce agreement on shared variables. Worst-case enumeration of the CSP solution space is exponential in the number of overlapping regions, and practical SPARQL-based consistency queries often time out at modest graph sizes (10^4 triples).

SheafDB consistency sheaf. SheafDB replaces CSP enumeration with the sheaf-theoretic cocycle condition. A *consistency sheaf* assigns to each context U the set $\mathcal{F}(U)$ of tuples valid in U, with restriction maps $\rho_{U,V} : \mathcal{F}(U) \to \mathcal{F}(V)$ for $V \subseteq U$. Global consistency holds when, for every pair of overlapping contexts U, V and every tuple $t \in \mathcal{F}(U \cap V)$, the cocycle condition

$$\rho_{U\cap V,W} \circ \rho_{U,U\cap V}(t) = \rho_{U\cap V,W} \circ \rho_{V,U\cap V}(t)$$

is satisfied for all $W \subseteq U \cap V$. In practice this reduces to checking that restriction maps agree on the intersection—a polynomial-time comparison of $O(|\mathcal{F}(U \cap V)|)$ tuples per overlapping pair.

Global section construction. Given a consistent family of local assignments, the *global section*—a single consistent fact assignment across the entire context space—is constructed via the gluing axiom. SheafDB implements this as a bottom-up merge over the context lattice: for each maximal chain of nested contexts, local sections are composed via restriction maps. The construction completes in $O(m \cdot d)$ where m is the number of contexts and d is the lattice depth.

Applications. Global sections enable principled data integration: heterogeneous sources produce local sections of the same entity sheaf, and the gluing axiom determines whether they can be merged without contradiction. In sensor fusion, overlapping sensor coverage regions are contexts; the consistency sheaf guarantees that fused readings satisfy physical continuity constraints.

The measured overhead of SheafDB's consistency checking (C8) and global section construction (C9) appears in Section 15. Complexity analysis for the CSP baseline is given in Section 6.

20 Ablation Studies

To identify which components of SheafDB contribute most to end-to-end performance, we conduct a scoped ablation comparing three configuration profiles that span the practical optimization space:

- **Minimal** — core tuple storage only; no caching, no indexing beyond the primary open-set index.
- **Stalk-indexed** — default SheafDB with stalk indexing, restriction-map caching, and temporal index enabled.
- **Full** — all available optimisations including provenance morphisms, interval indexing, and global section caching.

Each profile is evaluated on the representative workloads from Sections 16 through 19 at 10K-fact scale. The per-profile latency breakdown mirrors the LOOKUP/GLOBAL decomposition reported in Section 15, with stalk indexing providing the dominant contribution on entity-centric lookups and the full profile adding marginal benefit on mixed workloads.

We leave a comprehensive 33-dimensional leave-one-out analysis—including the pairwise interaction sweeps and sensitivity surfaces described in the companion optimisation framework—for future work. The modular architecture of the SheafDB engine (Section 7) is designed to support such fine-grained ablation when larger-scale ground-truth data become available.

21 Mixed Flagship Benchmark (C10) and Overall Results

The flagship workload. Benchmark C10 is the primary end-to-end evaluation workload, designed to exercise all layers of SheafDB simultaneously in a realistic multi-stage pipeline. It models a biomedical knowledge discovery scenario: given a set of seed entities, retrieve their interactors (neighborhood), filter by temporal validity (temporal), trace the provenance chain of each filtered fact (provenance), and verify global consistency across overlapping annotation sources (consistency).

Multi-stage pipeline. The pipeline proceeds in four stages, each corresponding to one of the targeted workload families:

1. **Neighborhood (C1/C2)** — retrieve all tuples mentioning any seed entity, with predicate filtering.

2. **Temporal filter (C5)** — restrict to tuples whose validity window intersects a query interval.

3. **Provenance trace (C7)** — walk the derivation chain for each surviving tuple to depth d.

4. **Consistency check (C8)** — verify that the provenance DAG satisfies consistency conditions across all overlapping annotation contexts.

KG approach. A conventional RDF store must issue four separate SPARQL queries, each performing its own graph traversal. Intermediate results—sets of triple IDs—are materialized between stages and passed as SPARQL VALUES clauses or temporary tables.

SheafDB approach. SheafDB executes the full pipeline as a single optimized sheaf walk. The sheaf structure preserves entity locality (stage 1), temporal indices are consulted during the walk without materializing intermediate results (stage 2), provenance edges are followed via sheaf morphisms that share the walk state (stage 3), and consistency is verified incrementally as each new context is visited (stage 4). Shared state across stages avoids redundant computation.

Current status. The individual pipeline stages correspond to the per-workload benchmarks described in Sections 16 through 19. Per-stage LOOKUP and GLOBAL query breakdowns appear in Section 15. An integrated end-to-end pipeline that chains all four stages as a single benchmark is the subject of ongoing work; the present evaluation focuses on the per-stage characterisation that exposes the fundamental LOOKUP-vs-GLOBAL tradeoff.

22 Discussion

22.1 Interpretation of Results

The experimental results reveal a nuanced picture. SheafDB's stalk-indexed LOOKUP is orders of magnitude faster than KG's triple reconstruction, confirming that context-indexed section storage avoids join overhead for entity-centric queries. However, SheafDB's GLOBAL (full scan) performance is significantly worse because the current implementation iterates over all open sets rather than maintaining a flat fact index.

This tradeoff is inherent to the sheaf representation: organizing facts by context enables fast local queries but adds overhead for global operations. The paper's contribution is the approach itself — a mathematically rigorous framework for contextual knowledge representation — not a claim of universal performance superiority.

22.2 When SheafDB Excels

SheafDB is preferred when:

- Queries are entity-centric LOOKUPs, where the stalk index provides O(1) retrieval vs. KG's O(k) triple reconstruction.
- Facts have high arity ($k \geq 4$), where reification overhead becomes significant.
- Context-scoped queries benefit from restriction-map locality.
- Cross-engine verification is required for data integrity.

22.3 When KG Excels

The KG representation is preferred when:

- Full scans (GLOBAL queries) are the primary workload.
- The dataset is flat (single context), providing no locality benefit.
- Integration with existing RDF/SPARQL infrastructure is required.

22.4 Threats to Validity

- **Implementation bias**: Both engines were implemented by the same authors. The KG baseline may not reflect the performance of mature systems like Jena or Virtuoso.
- **Dataset bias**: Synthetic datasets may not capture all characteristics of real-world knowledge graphs.
- **Query bias**: The benchmark query set may favor one representation over another.
- **Scale**: Results at 10K facts are indicative but do not demonstrate production-scale behavior. Standard RDF benchmarks (LUBM, BSBM) start at 100K triples.

22.5 Negative Results

SheafDB's GLOBAL query performance degrades significantly with dataset size (1405× slower than KG at 10K facts). This is because the current implementation iterates over all open sets to collect unique facts, rather than maintaining a flat fact index. A production-quality implementation would add a global fact index to bypass open set iteration for full scans.

The sheaf condition (locality + gluing) is mathematically vacuous for the Alexandrov topology on a finite poset, where every presheaf is automatically a sheaf. The system is more accurately described as a presheaf database with optional consistency verification. The value lies in the context-indexed organization and cross-engine verification, not in the sheaf condition itself.

23 Limitations

23.1 Scalability Limits

The current implementation is single-node and in-memory. Global section computation requires $O(|\mathcal{C}| \cdot N^2)$ time in the worst case, which limits scalability to datasets where $|\mathcal{C}|$ and N are moderate (on the order of 10^4 facts). All results in this paper were obtained at scales up to 10,000 facts. Behaviour at larger scales is extrapolated from complexity analysis and is the subject of ongoing work.

23.2 Mathematical Assumptions

The sheaf model assumes that the context poset is finite and has the Alexandrov topology. This is appropriate for our setting but may not generalize to infinite or continuous context spaces. The gluing condition assumes compatible sections can always be identified, which may fail in the presence of contradictory information.

23.3 Implementation Limitations

The current prototype does not support:

- Concurrent access or transactions.
- Persistent storage (in-memory only).
- Distributed execution.
- Streaming or incremental fact ingestion.
- Full SPARQL compliance (custom query language).

23.4 Benchmark Limitations

The benchmark suite covers 15 query types. While representative, this is not exhaustive. Real-world workloads may exhibit different access patterns. The synthetic datasets, while controlled, may not capture all nuances of organic knowledge graph data.

24 Future Work

24.1 Distributed SheafDB

Extending SheafDB to distributed execution across multiple nodes would enable scaling to larger datasets. The sheaf structure naturally partitions by context subtree, suggesting a distributed design where each node owns a subset of contexts and communicates via restriction maps.

24.2 Parallel Query Execution

Global section computation and gluing are embarrassingly parallel at the fact level. Future work could exploit multi-threading and GPU acceleration for these operations.

24.3 Incremental Restriction Maps

Currently, restriction maps are computed on demand. Caching frequently used restrictions and incrementally updating the cache on fact insertion would improve query performance.

24.4 Dynamic Topologies

The context poset is currently static. Supporting dynamic insertion and deletion of contexts would enable applications where the scoping structure evolves over time.

24.5 Global Fact Index for Full-Scan Queries

The current implementation's GLOBAL query performance degrades to 1405× slower than KG at 10K facts due to open-set iteration. A flat fact index maintained in parallel with the sheaf structure would allow full-scan queries to bypass open-set enumeration entirely, matching KG performance while preserving LOOKUP advantages.

24.6 Compression

Context-indexed storage admits specialized compression schemes: sections sharing the same context can be packed and encoded together, reducing the storage overhead identified in the evaluation.

24.7 Hybrid Systems

A hybrid system that dynamically selects between KG and sheaf representations based on query patterns could combine the strengths of both approaches. The optimizer could decide at query time which engine to use, or data could be replicated across both representations with automatic synchronization.

25 Conclusion

We have presented a sheaf-theoretic approach to semantic fact storage and querying. Our results demonstrate that the sheaf representation provides significant advantages for entity-centric LOOKUP queries (0.003–0.074ms vs. 0.008–2.279ms for KG) while matching KG insert performance (0.88–0.96×). However, full-scan (GLOBAL) queries are substantially slower due to open set iteration overhead.

The formal model — based on presheaves over a finite poset of contexts with restriction maps and gluing — provides a rigorous foundation for contextual knowledge representation. The implementation demonstrates that this model can be realized as a working database system with verified cross-engine semantic equivalence.

Key findings:

1. SheafDB's stalk-indexed LOOKUP is 3–30× faster than KG across all tested scales.

2. Insert performance is comparable (SheafDB 0.88–0.96× KG).

3. GLOBAL (full scan) queries are 1–1400× slower, revealing a need for a flat fact index in production implementations.

4. Cross-engine semantic equivalence is verified: both engines produce identical canonical results for all query types.

5. The sheaf approach is best understood as a presheaf-based contextual knowledge store, not as a sheaf in the strict topological sense (the sheaf condition is vacuous for the Alexandrov topology on a finite poset).

The sheaf representation is preferred when queries are entity-centric and context-scoped. The traditional KG representation is preferred for full-scan workloads. A hybrid approach combining both representations — using the stalk index for LOOKUP and a flat triple index for scans — may offer the best of both worlds.

26 Artifact Availability

The complete source code, data, and experimental results are available at: `https://github.com/BlackSamuron0305/semantic-fact-db`

26.1 Reproducibility

The following information is automatically recorded for reproducibility:

Table 6: Reproducibility metadata.

Property	Value
Git commit	`fa42b64`
Python version	`3.14.6`
uv lock hash	`f5166dc09722c41f`
Dataset checksum	`auto`
Random seed	`42`
Generation date	`2026-07-08T21:48:52.504442`

26.2 License

27 Appendix

27.1 Proofs

Detailed proofs of all theorems are available in the repository at `research/proofs/`. These include:

- Presheaf axioms (Section 3.1.2).
- Sheaf condition (Section 3.1.3).
- Locality and gluing (Section 3.1.3).
- Global sections (Section 3.1.5).
- Restriction maps.
- Local sections.
- Stalks.
- Alexandrov topology.
- Simplicial model.

27.2 Algorithms

Pseudocode for all algorithms is available at `research/mathematics/algorithms.md`.

27.3 Validation

The implementation-to-definition validation table is available at `research/mathematics/validation.md`.

References

[1] Renzo Angles. The property graph database model. *Proceedings of the Alberto Mendelzon International Workshop on Foundations of Data Management*, 1912:1–12, 2017.

[2] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.

[3] Tim Berners-Lee, James Hendler, and Ora Lassila. The semantic web. *Scientific American*, 284(5):34–43, 2001.

[4] Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46(2):255–308, 2009.

[5] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

[6] Giorgio Gallo, Giustino Longo, and Stefano Pallottino. Directed hypergraphs and applications. *Discrete Applied Mathematics*, 42(2–3):177–201, 1993.

[7] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[8] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C recommendation, World Wide Web Consortium, 2013.

[9] Saunders Mac Lane and Ieke Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Universitext. Springer, 1992.

[10] Frank Manola and Eric Miller. RDF primer. W3C recommendation, World Wide Web Consortium, 2004.

[11] Neo Technology. Neo4j graph database, 2024.

[12] Evan Patterson. Sheaf theory in database theory. In *Proceedings of the 3rd Annual International Symposium on Category Theory and Computer Science*, 2022.

[13] Michael Robinson. Sheaf theory in data fusion. *SIAM Review*, 62(3), 2020.